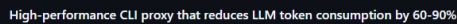


 Go to file
 
 Add file
  Code

[📖 README](#)
[👤 Contributing](#)
[📄 Apache-2.0 license](#)
[🔒 Security](#)
[✎](#)
[☰](#)



 Security Check passing
 release v0.42.4
 License Apache 2.0
 Discord 653 online
 homebrew v0.42.4

[English](#) • [Français](#) • [中文](#) • [日本語](#) • [한국어](#) • [Español](#) • [Português](#)

About

www.rtk-ai.app

v0.42.4 **Latest**
last week

● Rust 92.9%
 ● Shell 4.8%
 ● TypeScript 1.5%
 ● Other 0.8%

Token Savings (30-min Claude Code Session)

Operation	Frequency	Standard	rtk	Savings
ls / tree	10x	2,000	400	-80%
cat / read	20x	40,000	12,000	-70%
grep / rg	8x	16,000	3,200	-80%
git status	10x	3,000	600	-80%
git diff	5x	10,000	2,500	-75%
git log	5x	2,500	500	-80%
git add/commit/push	8x	1,600	120	-92%
cargo test / npm test	5x	25,000	2,500	-90%
ruff check	3x	3,000	600	-80%
pytest	4x	8,000	800	-90%
go test	3x	6,000	600	-90%
docker ps	3x	900	180	-80%
Total		~118,000	~23,900	-80%

Estimates based on medium-sized TypeScript/Rust projects. Actual savings vary by project size.

Installation

Homebrew (recommended)

```
brew install rtk
```

Quick Install (Linux/macOS)

```
curl -fsSL https://raw.githubusercontent.com/rtk-ai/rtk/refs/heads/master/install.sh | sh
```

```
Installs to ~/.local/bin. Add to PATH if needed:

echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc # or ~/.zshrc
```

Cargo

```
cargo install --git https://github.com/rtk-ai/rtk
```

Pre-built Binaries

Download from [releases](#):

- macOS: [rtk-x86_64-apple-darwin.tar.gz](#) / [rtk-aarch64-apple-darwin.tar.gz](#)
- Linux: [rtk-x86_64-unknown-linux-musl.tar.gz](#) / [rtk-aarch64-unknown-linux-gnu.tar.gz](#)
- Windows: [rtk-x86_64-pc-windows-msvc.zip](#)

Windows users: Extract the zip and place `rtk.exe` somewhere in your PATH (e.g. `C:\Users\you\local\bin`). Run RTK from Command Prompt, PowerShell, or Windows Terminal — do not double-click the `.exe` (it will flash and close). For the best experience, use [WSL](#) where the full hook system works natively. See [Windows setup](#) below for details.

Verify Installation

```
rtk --version # Should show "rtk 0.28.2"
rtk gain      # Should show token savings stats
```

Name collision warning: Another project named "rtk" (Rust Type Kit) exists on crates.io. If `rtk gain` fails, you have the wrong package. Use `cargo install --git` above instead.

Quick Start

```
# 1. Install for your AI tool
rtk init -g # Claude Code / Copilot (default)
rtk init -g --gemini # Gemini CLI
rtk init -g --codex # Codex (OpenAI)
rtk init -g --agent cursor # Cursor
rtk init -g --agent windsurf # Windsurf
rtk init --agent cline # Cline / Roo Code
rtk init --agent kilocode # Kilo Code
rtk init --agent antigravity # Google AntigraVity
rtk init -g --agent pi # Pi
rtk init --agent hermes # Hermes

# 2. Restart your AI tool, then test
git status # Automatically rewritten to rtk git status
```

Hook-based agents rewrite Bash commands (e.g., `git status -> rtk git status`) before execution. Plugin-based agents, including Hermes, use their plugin API to rewrite commands before execution. The agent receives compact output without needing to call `rtk` explicitly.

Important: the hook only runs on Bash tool calls. Claude Code built-in tools like `Read`, `Grep`, and `Glob` do not pass through the Bash hook, so they are not auto-rewritten. To get RTK's compact output for those workflows, use shell commands (`cat / head / tail`, `rg / grep`, `find`) or call `rtk read`, `rtk grep`, or `rtk find` directly.

How It Works

Without rtk:	With rtk:
Claude --git status--> shell --> git ^ ~2,000 tokens (raw) +-----+	Claude --git status--> RTK --> git ^ filter ~200 tokens +----- (filtered) -----+

Four strategies applied per command type:

1. **Smart Filtering** - Removes noise (comments, whitespace, boilerplate)
2. **Trimming** - Truncates trailing spaces/lines by default (optional)

- 2. **Grouping** - Aggregates similar items (files by directory, errors by type)
- 3. **Truncation** - Keeps relevant context, cuts redundancy
- 4. **Deduplication** - Collapses repeated log lines with counts

Commands

Files

rtk ls .	# Token-optimized directory tree
rtk read file.rs	# Smart file reading
rtk read file.rs -l aggressive	# Signatures only (strips bodies)
rtk smart file.rs	# 2-line heuristic code summary
rtk find "*.rs" .	# Compact find results
rtk grep "pattern" .	# Grouped search results
rtk diff file1 file2	# Condensed diff (exit 1 if files differ)

Git

rtk git status	# Compact status
rtk git log -n 10	# One-line commits
rtk git diff	# Condensed diff
rtk git add	# -> "ok"
rtk git commit -m "msg"	# -> "ok abc1234"
rtk git push	# -> "ok main"
rtk git pull	# -> "ok 3 files +10 -2"

GitHub CLI

rtk gh pr list	# Compact PR listing
rtk gh pr view 42	# PR details + checks
rtk gh issue list	# Compact issue listing
rtk gh run list	# Workflow run status

Test Runners

rtk jest	# Jest compact (failures only)
rtk vitest	# Vitest compact (failures only)
rtk playwright test	# E2E results (failures only)
rtk pytest	# Python tests (-90%)
rtk go test	# Go tests (NDJSON, -90%)
rtk cargo test	# Cargo tests (-90%)
rtk rake test	# Ruby minitest (-90%)
rtk rspec	# RSpec tests (JSON, -60%+)
rtk err <cmd>	# Filter errors only from any command
rtk test <cmd>	# Generic test wrapper - failures only (-90%)

Build & Lint

rtk lint	# ESLint grouped by rule/file
rtk lint blome	# Supports other linters
rtk tsc	# TypeScript errors grouped by file
rtk next build	# Next.js build compact
rtk prettier --check .	# Files needing formatting
rtk cargo build	# Cargo build (-80%)
rtk cargo clippy	# Cargo clippy (-80%)
rtk ruff check	# Python linting (JSON, -80%)
rtk golangci-lint run	# Go linting (JSON, -85%)
rtk rubocop	# Ruby linting (JSON, -60%+)

Package Managers

rtk npm list	# Compact dependency tree
rtk pip list	# Python packages (auto-detect uv)
rtk pip outdated	# Outdated packages
rtk bundle install	# Ruby gems (strip Using lines)
rtk prisma generate	# Schema generation (no ASCII art)

AWS

rtk aws sts get-caller-identity	# One-line identity
rtk aws ec2 describe-instances	# Compact instance list
rtk aws lambda list-functions	# Name/runtime/memory (strips secrets)
rtk aws logs get-log-events	# Timestamped messages only
rtk aws cloudformation describe-stack-events	# Failures first
rtk aws dynamodb scan	# Unwraps type annotations
rtk aws iam list-roles	# Strips policy documents
rtk aws s3 ls	# Truncated with tee recovery

Containers

rtk docker ps	# Compact container list
rtk docker images	# Compact image list
rtk docker logs <container>	# Deduplicated logs
rtk docker compose ps	# Compose services
rtk kubectl pods	# Compact pod list
rtk kubectl logs <pod>	# Deduplicated logs
rtk kubectl services	# Compact service list

Data & Analytics

rtk json config.json	# Structure without values
rtk deps	# Dependencies summary
rtk env -f AWS	# Filtered env vars
rtk log app.log	# Deduplicated logs
rtk curl <url>	# Truncate + save full output
rtk wget <url>	# Download, strip progress bars
rtk summary <long command>	# Heuristic summary
rtk proxy <command>	# Raw passthrough + tracking

Token Savings Analytics

rtk gain	# Summary stats
rtk gain --graph	# ASCII graph (last 30 days)
rtk gain --history	# Recent command history
rtk gain --daily	# Day-by-day breakdown
rtk gain --all --format json	# JSON export for dashboards
rtk discover	# Find missed savings opportunities

```
rtk discover --all --since 7 # All projects, last 7 days

rtk session # Show RTK adoption across recent sessions
```

Global Flags

```
-u, --ultra-compact # ASCII icons, inline format (extra token savings)
-v, --verbose # Increase verbosity (-V, -VV, -VVV)
```

Examples

Directory listing:

```
# ls -la (45 lines, ~800 tokens) # rtk ls (12 lines, ~150 tokens)
drwxr-xr-x 15 user staff 480 ... my-project/
-rw-r--r-- 1 user staff 1234 ... +-- src/ (8 files)
... | +-- main.rs
+-- Cargo.toml
```

Git operations:

```
# git push (15 lines, ~200 tokens) # rtk git push (1 line, ~10 tokens)
Enumerating objects: 5, done. ok main
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
```

Test output:

```
# cargo test (200+ lines on failure) # rtk test cargo test (~20 lines)
running 15 tests FAILED: 2/15 tests
test utils::test_parse ... ok test_edge_case: assertion failed
test utils::test_format ... ok test_overflow: panic at utils.rs:18
...
```

Auto-Rewrite Hook

The most effective way to use rtk. The hook transparently intercepts Bash commands and rewrites them to rtk equivalents before execution.

Result: 100% rtk adoption across all conversations and subagents, zero token overhead.

Scope note: this only applies to Bash tool calls. Claude Code built-in tools such as `Read`, `Grep`, and `Glob` bypass the hook, so use shell commands or explicit `rtk` commands when you want RTK filtering there.

Setup

```
rtk init -g # Install hook + RTK.md (recommended)
rtk init -g --opencode # OpenCode plugin (instead of Claude Code)
rtk init -g --auto-patch # Non-interactive (CI/CD)
rtk init -g --hook-only # Hook only, no RTK.md
rtk init --show # Verify installation
```

After install, restart Claude Code.

Windows

RTK works on Windows with some limitations. The auto-rewrite hook (`rtk-rewrite.sh`) requires a Unix shell, so on native Windows RTK falls back to **CLAUDE.md injection mode** — your AI assistant receives RTK instructions but commands are not rewritten automatically.

Recommended: WSL (full support)

For the best experience, use [WSL](#) (Windows Subsystem for Linux). Inside WSL, RTK works exactly like Linux — full hook support, auto-rewrite, everything:

```
# Inside WSL
curl -fsSL https://raw.githubusercontent.com/rtk-ai/rtk/refs/heads/master/install.sh | sh
rtk init -g
```

Native Windows (limited support)

On native Windows (`cmd.exe` / PowerShell), RTK filters work but the hook does not auto-rewrite commands:

```
# 1. Download and extract rtk-x86_64-pc-windows-msvc.zip from releases
# 2. Add rtk.exe to your PATH
# 3. Initialize (falls back to CLAUDE.md injection)
rtk init -g
# 4. Use rtk explicitly
rtk cargo test
rtk git status
```

Important Do not double-click `rtk.exe` — it is a CLI tool that prints usage and exits immediately. Always run it from a terminal (Command Prompt, PowerShell, or Windows Terminal).

Feature	WSL	Native Windows
Filters (cargo, git, etc.)	Full	Full
Auto-rewrite hook	Yes	No (CLAUDE.md fallback)
<code>rtk init -g</code>	Hook mode	CLAUDE.md mode
<code>rtk gain</code> / analytics	Full	Full

Supported AI Tools

RTK supports 14 AI coding tools. Each integration rewrites shell commands to `rtk` equivalents for 60-90% token savings where the agent supports command interception.

Tool	Install	Method
Claude Code	<code>rtk init -g</code>	PreToolUse hook (bash)
GitHub Copilot (VS Code)	<code>rtk init -g --copilot</code>	PreToolUse hook — transparent rewrite
GitHub Copilot CLI	<code>rtk init -g --copilot</code>	PreToolUse deny-with-suggestion (CLI limitation)

Cursor	<code>rtk init -g --agent cursor</code>	preToolUse hook (hooks.json)
Gemini CLI	<code>rtk init -g --gemini</code>	BeforeTool hook
Codex	<code>rtk init -g --codex</code>	AGENTS.md + RTK.md instructions
Windsurf	<code>rtk init -g --agent windsurf</code>	.windsurfrules (project-scoped)
Cline / Roo Code	<code>rtk init --agent cline</code>	.clinerules (project-scoped)
OpenCode	<code>rtk init -g --opencode</code>	Plugin TS (tool.execute.before)
OpenClaw	<code>openclaw plugins install</code> <code>./openclaw</code>	Plugin TS (before_tool_call)
Pi	<code>rtk init -g --agent pi</code> (global)	TypeScript extension (tool_call)
Hermes	<code>rtk init --agent hermes</code>	Python plugin adapter (terminal command mutation via <code>rtk rewrite</code>)
Mistral Vibe	Planned (#800)	Blocked on upstream
Kilo Code	<code>rtk init --agent kilocode</code>	.kilocode/rules/rtk-rules.md (project-scoped)
Google Antigravity	<code>rtk init --agent antigravity</code>	.agents/rules/antigravity-rtk-rules.md (project-scoped)

For per-agent setup details, override controls, and graceful degradation, see the [Supported Agents guide](#). The Hermes plugin source and tests live in `hooks/hermes/`; installed Hermes runtime files still live under `~/hermes/plugins/rtk-rewrite/`.

Configuration

`~/config/rtk/config.toml` (macOS: `~/Library/Application Support/rtk/config.toml`):

```
[hooks]
exclude_commands = ["curl", "playwright"] # skip rewrite for these

[tee]
enabled = true           # save raw output on failure (default: true)
mode = "failures"       # "failures", "always", or "never"
```

When a command fails, RTK saves the full unfiltered output so the LLM can read it without re-executing:

```
FAILED: 2/15 tests
[full output: ~/local/share/rtk/tee/1707753600_cargo_test.log]
```

For the full config reference (all sections, env vars, per-project filters), see the [Configuration guide](#).

Uninstall

```
rtk init -g --uninstall # Remove hook, RTK.md, settings.json entry
cargo uninstall rtk    # Remove binary
brew uninstall rtk     # If installed via Homebrew
```

Documentation

- [rtk-ai.app/guide](#) — full user guide (installation, supported agents, what gets optimized, analytics, configuration, troubleshooting)
- [INSTALL.md](#) — detailed installation reference
- [ARCHITECTURE.md](#) — system design and technical decisions
- [CONTRIBUTING.md](#) — contribution guide
- [SECURITY.md](#) — security policy

Privacy & Telemetry

RTK can collect **anonymous, aggregate usage metrics** once per day. Telemetry is **disabled by default** and requires **explicit opt-in consent** (GDPR Art. 6, 7) during `rtk init` or via `rtk telemetry enable`. This data helps us build a better product: identifying which commands need filters, which filters need improvement, and how much value RTK delivers. For the full list of fields, data handling, and contributor guidelines, see [docs/TELEMETRY.md](#).

What is collected and why:

Category	Data	Why
Identity	Salted device hash (SHA-256, not reversible)	Count unique installations without tracking individuals
Environment	RTK version, OS, architecture, install method	Know which platforms to support and test
Usage volume	Command count (24h), total commands, tokens saved (24h/30d/total)	Measure adoption and value delivered
Quality	Top 5 passthrough commands (0% savings), parse failure count, commands with <30% savings	Identify missing filters and weak ones to improve
Ecosystem	Command category distribution (e.g. git 45%, cargo 20%, js 15%)	Prioritize filter development for popular ecosystems
Retention	Days since first use, active days in last 30	Understand engagement and detect churn
Adoption	AI agent hook type (claude/gemini/codex), custom TOML filter count	Track integration coverage and DSL adoption
Configuration	Whether config.toml exists, number of excluded commands, project count	Understand user maturity and customization patterns
Features	Usage counts for meta-commands (gain, discover, proxy, verify)	Know which RTK features are valued vs unused
Economics	Estimated USD savings (based on API token pricing)	Quantify the value RTK provides to users

All data is **aggregate counts** or **anonymized command names** (first 3 words, no arguments). Top commands report only tool names (e.g. "git", "cargo"), never full command lines.

What is NOT collected: source code, file paths, command arguments, secrets, environment variables, personal data, or repository contents.

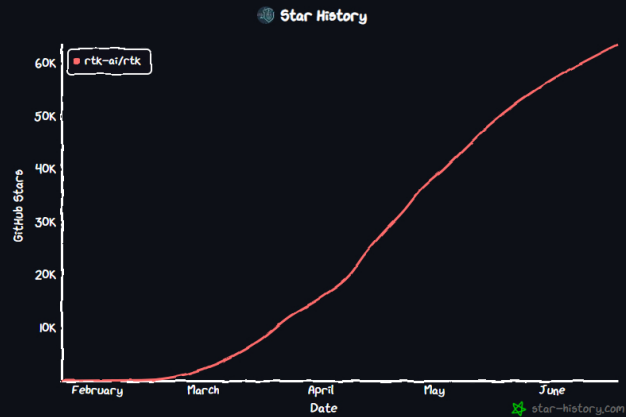
Manage telemetry:

```
rtk telemetry status # Check current consent state
rtk telemetry enable # Give consent (interactive prompt)
rtk telemetry disable # Withdraw consent - stops all collection immediately
rtk telemetry forget # Withdraw consent + delete all local data + request server-side erasure
```

Override via environment:

```
export RTK_TELEMETRY_DISABLED=1 # Blocks telemetry regardless of consent
```

Star History



StarMapper



Core team

- Patrick Szymkowiak — Founder [GitHub](#) · [LinkedIn](#)
- Florian Bruniaux — Core contributor [GitHub](#) · [LinkedIn](#)
- Adrien Eppling — Core contributor [GitHub](#) · [LinkedIn](#)

Contributing

Contributions welcome! Please open an issue or PR on [GitHub](#).

Join the community on [Discord](#).

License

Apache License 2.0 - see [LICENSE](#) for details.

Disclaimer

See [DISCLAIMER.md](#).